MOBILE APPLICATION SECURITY AUDIT

# Security Audit Report – Clozr (Flutter mobile CRM, `com.nexotech.clozrapp`)

| | |
|---|---|
| Date | 2026-10-27 |
| Auditor | Claude (automated static review – read-only, no code changed) |
| Scope | `lib/**`, `test/**`, `pubspec.yaml`, `pubspec.lock`, `assets/**`, `android/**` (manifests, Gradle, Kotlin), `ios/**` (Info.plist, AppDelegate, Podfile, pbxproj), `web/index.html`, `.gitignore`, `.flutter-plugins`, `README.md`, `docs-flutter/**`, and the reachable git history of branch `fix/uat-crm-common-parity` (50 commits, single branch in this clone) |
| Standards | OWASP Mobile Top 10 (2024), OWASP MASVS v2, CWE |
| App version audited | `1.0.0+10` |

| 0 | 1 | 3 | 7 | 4 |
|---|---|---|---|---|
| CRITICAL | HIGH | MEDIUM | LOW | INFO |

## Contents

1. Executive Summary
2. Findings Summary Table
3. Detailed Findings
4. Needs Manual Verification
5. Dependency Review
6. Positive Findings
7. Prioritized Remediation Roadmap
8. Appendix

## 1. Executive Summary

- **Overall risk level: Medium.** No hardcoded third-party secret, no open BaaS rule set, no disabled TLS validation and no payment bypass exists in this codebase. The app is a thin client over a Django REST backend ( `/api/v1` ), with JWT auth, 2FA, token storage in Keystore/Keychain, and a well-scoped Sentry integration. The problems are configuration and data-handling gaps rather than classic "key in the APK" issues.
- **A release build silently talks to the development server.** The backend address defaults to `https://dev.clozr.tech` and the production address only appears as a comment. There is no CI, build script or documented release flag, so whoever forgets `--dart-define=API_BASE_URL=…` ships customers' logins and CRM data to the dev environment. The previous in-repo audit found exactly this already compiled into a release bundle, and the default is still there.
- **Customer data is cached on disk unencrypted and is eligible for Android cloud backup.** Leads, customers, invoices, payments, WhatsApp message text, tickets, and the signed-in user's profile sit in plain Hive boxes, and the Android manifest does not opt out of backup. A B2B tenant's records can end up in an employee's personal Google account and are readable on a rooted or forensically imaged device.
- **The app trusts server-supplied strings in two places where it should not:** a WhatsApp `media_id` becomes a filename without sanitisation (path traversal inside the app sandbox), and an attachment URL is handed to `launchUrl` with any scheme it carries (so a hostile or compromised backend could trigger `intent:` / `file:` / `sms:` style launches instead of a browser).
- **Nothing in the repo makes the release build hard to reverse-engineer:** no `--obfuscate --split-debug-info` , no R8 minification, and no release checklist. For a CRM this is a hardening gap, not a breach, but it makes finding #1's dev URL and all API routes trivially extractable with `strings` .

## 2. Findings Summary Table

| ID | Title | Severity | Category (OWASP M#) | MASVS | File(s) |
|---|---|---|---|---|---|
| FSA-001 | Release build defaults to the development backend; production URL is a comment | High | M8 Security Misconfiguration | MASVS-NETWORK-1 | `lib/core/config/api_config.dart:22-28` |
| FSA-002 | Unencrypted Hive cache of tenant CRM data, eligible for Android auto-backup | Medium | M9 Insecure Data Storage | MASVS-STORAGE-1/2 | `lib/core/storage/app_cache.dart` , `android/app/src/main/AndroidManifest.xml:15-19` |
| FSA-003 | Server-supplied `media_id` used verbatim as a filename (path traversal in sandbox) | Medium | M4 Insufficient Input/Output Validation | MASVS-CODE-4 | `lib/features/messages/presentation/screens/chat_screen.dart:220-222` |
| FSA-004 | Attachment/recording URLs launched with any scheme the server returns | Medium | M4 / M8 | MASVS-PLATFORM-1 | `lib/core/utils/attachment_link.dart:21-32,53-66` |
| FSA-005 | No code obfuscation, no release build pipeline or documented release flags | Low | M7 Insufficient Binary Protections | MASVS-RESILIENCE-3 | repo root (no CI), `README.md` , `android/app/build.gradle.kts:80-105` |
| FSA-006 | R8/ProGuard minification not enabled for the release build type | Low | M7 | MASVS-RESILIENCE-3 | `android/app/build.gradle.kts:80-105` |
| FSA-007 | Debug Dio logger dumps full response bodies, including the JWT pair | Low | M6 Inadequate Privacy Controls | MASVS-STORAGE-2 | `lib/core/network/api_service.dart:39-54` |
| FSA-008 | Client-generated `user_id` for invites uses non-cryptographic `Random()` | Low | M10 Insufficient Cryptography | MASVS-CRYPTO-1 | `lib/features/people/infrastructure/data_sources/remote/people_remote_ds.dart:155-156,617-619` |
| FSA-009 | 2FA backup codes and OTP entry shown without screenshot/app-switcher protection | Low | M6 | MASVS-PLATFORM-3 | `lib/features/auth/presentation/screens/login_screen.dart:385-412` |
| FSA-010 | HTTPS-only enforcement is an `assert` , stripped from release builds | Low | M5 Insecure Communication | MASVS-NETWORK-1 | `lib/app/bootstrap/app_bootstrap.dart:47-53` |

| ID | Title | Severity | Category (OWASP M#) | MASVS | File(s) |
|---|---|---|---|---|---|
| FSA-011 | Custom-scheme deep link ( `clozr://app` ) is hijackable and unverified; not declared on iOS | Low | M8 | MASVS-PLATFORM-1 | `android/app/src/main/AndroidManifest.xml:41-50` , `ios/Runner/Info.plist` |
| FSA-012 | Sentry DSN hardcoded; request URL + query string sent on 5xx | Info | M6 | MASVS-PRIVACY-1 | `lib/core/monitoring/sentry_config.dart:31-35,114-130` |
| FSA-013 | Raw exception text surfaced to users in toasts | Info | M6 | MASVS-PLATFORM-3 | `lib/features/crm/presentation/screens/quote_detail_screen.dart:485` and 9 similar sites |
| FSA-014 | Generated `.flutter-plugins` committed with a developer's home-directory path | Info | M8 | – | `.flutter-plugins:1-3` |
| FSA-015 | `hive` 2.2.3 is upstream-abandoned; `open_file` 3.x legacy manifest behaviour | Info | M2 Inadequate Supply Chain Security | MASVS-CODE-3 | `pubspec.yaml` , `pubspec.lock` |

**Totals:** Critical 0 · High 1 · Medium 3 · Low 7 · Info 4

## 3. Detailed Findings

### FSA-001 – Release build defaults to the development backend; production URL is a comment

- **Severity:** High – one forgotten flag routes every real customer's credentials and CRM records to a non-production environment; no tooling in the repo prevents it.
- **OWASP Mobile Top 10:** M8 – Security Misconfiguration
- **MASVS:** MASVS-NETWORK-1
- **CWE:** CWE-1188 (Insecure Default Initialization of Resource), CWE-15 (External Control of System or Configuration Setting)
- **Location:** `lib/core/config/api_config.dart:22-28`
- **Evidence:**

```
static const String baseUrl = String.fromEnvironment(
  'API_BASE_URL',
  defaultValue: 'https://dev.clozr.tech',
);

// Production backend:
// 'https://app.clozr.co.in'
```

  No CI configuration exists ( `.github/` , `codemagic.yaml` , `bitrise.yml` , `fastlane/` , `.gitlab-ci.yml` all absent). `README.md:41-58` documents the Sentry `--dart-define` s and a release build command but never mentions `API_BASE_URL` . `SentryConfig.environment` ( `lib/core/monitoring/sentry_config.dart:41-48` ) defaults to `production` in release mode, so a dev-pointed release build would also report into the production Sentry stream, masking the mistake.
- **Description:** The compile-time default is the UAT/dev host. Mock mode is only entered when the value is empty, so the "safe" fallback is to talk to dev, not to refuse. Because the app treats whatever host it is given as fully trusted (bearer tokens attached by `AuthInterceptor.onRequest` , data cached, files opened), the dev environment receives production logins, 2FA codes, refresh tokens and tenant data.
- **Attack Scenario:** (1) A developer runs `fvm flutter build appbundle --release` without the define (the README's own example omits it). (2) The AAB is uploaded to Play. (3) Every user authenticates against `dev.clozr.tech` ; their credentials, sessions and CRM data are now held by an environment with weaker access controls, test data, and different retention. (4) Anyone with dev-environment access, or an attacker who compromises the lower-security dev host, reads production tenant data and can mint sessions with the stored refresh tokens.
- **Impact:** All tenants and users of a mis-built release; confidentiality of credentials and customer records; compliance exposure for a B2B data processor.
- **Recommended Fix (not applied):**

```
// api_config.dart
static const String baseUrl = String.fromEnvironment('API_BASE_URL'); // no default

// app_bootstrap.dart — fail closed in release, not with an assert
if (kReleaseMode && ApiConfig.baseUrl.isEmpty) {
  throw StateError('API_BASE_URL must be supplied for release builds');
}
```

Add a committed release script (e.g. `tool/build_release.sh`) that hardcodes `--dart-define=API_BASE_URL=https://app.clozr.co.in --dart-define=SENTRY_ENVIRONMENT=production --obfuscate --split-debug-info=build/symbols`, and a UAT variant that passes the dev URL and `SENTRY_ENVIRONMENT=development`. Consider `--dart-define-from-file=env/prod.json`. Delete any previously built `build/` artefacts before upload.

- **References:** OWASP MASTG-TEST-0003 (App Configuration); the repo's own `SECURITY_AUDIT.md` §1 which found `https://dev.clozr.tech` inside a real `app-release.aab`.

### FSA-002 – Unencrypted Hive cache of tenant CRM data, eligible for Android auto-backup

- **Severity:** Medium – readable without root via Android auto-backup/ `adb backup` on older APIs, and on any rooted/jailbroken or forensically imaged device; requires device or account access rather than a remote attack.
- **OWASP Mobile Top 10:** M9 – Insecure Data Storage
- **MASVS:** MASVS-STORAGE-1, MASVS-STORAGE-2
- **CWE:** CWE-312 (Cleartext Storage of Sensitive Information), CWE-530 (Exposure of Backup File)
- **Location:** `lib/core/storage/app_cache.dart:43-60`; `android/app/src/main/AndroidManifest.xml:15-19`; `lib/features/auth/application/providers/auth_providers.dart:242-245`
- **Evidence:**

```
// app_cache.dart:43-48 — no HiveAesCipher / encryptionCipher anywhere
static Future<void> init() async {
  if (_ready) return;
  await Hive.initFlutter();
  await Future.wait(_boxes.map((name) => Hive.openBox<String>(name)));
```

```
// auth_providers.dart:245 — full profile (email, phone, roles, org) into the plain 'auth' box
AppCache.put(AppCache.authBox, _userCacheKey, jsonEncode(user.toJson()));
```

```
<!-- AndroidManifest.xml:15-19 — no android:allowBackup, no dataExtractionRules / fullBackupContent -->
<application
    android:label="Clozr"
    android:name="${applicationName}"
    android:icon="@mipmap/ic_launcher"
    android:usesCleartextTraffic="false">
```

`grep -rn "AppCache.put(" lib` shows 29 call sites writing raw API rows for leads, customers, payments, invoices, quotes, call logs, attachments, follow-ups, tasks, tickets, projects, accounting documents, notifications, WhatsApp conversations and message bodies, and people/teams. `grep -rnE "HiveAesCipher|encryptionCipher" lib` returns nothing.

- **Description:** Every list and detail the app shows is persisted as plaintext JSON under the app's documents directory. `allowBackup` defaults to `true`, so on Android 6+ the whole Hive directory is swept into Google's device backup of the signed-in *personal* Google account. JWTs are **not** affected (they live in `flutter_secure_storage`, whose Keystore key does not survive restore), but the data the tokens protect is.
- **Attack Scenario:** (1) An employee signs into Clozr on a personal phone with their personal Google account. (2) Android backs up `app_flutter/*.hive` to that account's Drive. (3) The employee leaves, or the personal account is phished; the tenant's customer list, payment records and WhatsApp threads are now outside the customer's control. Alternatively: a lost/stolen rooted device, or `adb backup` on API ≤ 30, yields the same files, parseable with any Hive reader or even `strings`.
- **Impact:** Confidentiality of every tenant's CRM/finance/messaging data cached on the device; GDPR/DPDP processor obligations.
- **Recommended Fix (not applied):**

1. Opt out of backup and exclude Hive even from device-to-device transfer:

```
<application ... android:allowBackup="false"
    android:fullBackupContent="@xml/backup_rules"
    android:dataExtractionRules="@xml/data_extraction_rules">
```

```
<!-- res/xml/data_extraction_rules.xml -->
<data-extraction-rules>
  <cloud-backup><exclude domain="file" path="."/><exclude domain="sharedpref" path="."/></cloud-backup>
  <device-transfer><exclude domain="file" path="."/><exclude domain="sharedpref" path="."/></device-transfer>
</data-extraction-rules>
```

2. Encrypt the boxes with a key kept in secure storage:

```
final stored = await secure.read(key: 'hive_key');
final key = stored == null ? Hive.generateSecureKey() : base64Url.decode(stored);
if (stored == null) await secure.write(key: 'hive_key', value: base64Url.encode(key));
await Hive.openBox<String>(name, encryptionCipher: HiveAesCipher(key));
```

3. Consider not caching the `auth` profile at all, or caching only `id` / `name`.

- **References:** MASTG-TEST-0001 (Testing Local Storage), MASTG-TEST-0009 (Backups); Android "Back up user data" docs; Hive encryption docs.

### FSA-003 – Server-supplied `media_id` used verbatim as a filename (path traversal inside the sandbox)

- **Severity:** Medium – exploitable only by a malicious or compromised backend (TLS protects the path), but then a single tap overwrites arbitrary files in the app's private storage. Rated Medium rather than High because the attacker must already control the API, and the writable target is the app's own sandbox.
- **OWASP Mobile Top 10:** M4 – Insufficient Input/Output Validation
- **MASVS:** MASVS-CODE-4
- **CWE:** CWE-22 (Path Traversal), CWE-73 (External Control of File Name)
- **Location:** `lib/features/messages/presentation/screens/chat_screen.dart:212-225`; source of value: `lib/features/messages/infrastructure/data_sources/remote/messages_remote_ds.dart:139,159`; also used in the request path at `lib/core/network/api_endpoints.dart:283`
- **Evidence:**

```
// chat_screen.dart:219-222
final (bytes, contentType) = fetched;
final dir = await getTemporaryDirectory();
final file = File('${dir.path}/wa_$mediaId${_extensionForMime(contentType)}');
await file.writeAsBytes(bytes);
```

```
// messages_remote_ds.dart:139 — straight from JSON, no validation
mediaId: (type == 'image' || type == 'document') ? _str(row['media_id']) : null,
```

```
// api_endpoints.dart:283
static String whatsappMedia(Object mediaId) => '/whatsapp/media/$mediaId/';
```

Grepping every hop (`chatMessageFromJson` → `ChatMessage.mediaId` → `fetchMedia` → `fetchMediaBytes` → `_fetchAndOpenMedia`) finds no character allow-list or `path.basename` call. Contrast with the quote PDF path at `quote_detail_screen.dart:474`, which *does* sanitise: `.replaceAll(RegExp(r'[^A-Za-z0-9_-]'), '_')`.

- **Description:** `media_id` is whatever the `/whatsapp/conversations/{id}/messages/` endpoint returns. A value such as `../app_flutter/crm_cache.hive` escapes the cache directory. On Android the temp dir is `/data/data/com.nexotech.clozrapp/cache`, so `../app_flutter/` (Hive) and `../shared_prefs/` (flutter_secure_storage's EncryptedSharedPreferences file) are reachable. The same unescaped value is also interpolated into the request path, so it can also redirect the authenticated GET to another API route.
- **Attack Scenario:** (1) Attacker controls or compromises the backend (or a tenant's WhatsApp integration writes attacker-chosen `media_id` values into message rows, if the server stores them unvalidated – see §4). (2) User taps an attachment in chat. (3) The app writes attacker-supplied bytes to an attacker-chosen path inside its sandbox – corrupting the Hive cache (DoS), replacing the secure-storage prefs file (forced logout), or planting a file the OS viewer then opens.
- **Impact:** Integrity/availability of local app data; limited confidentiality impact (no read primitive). Multi-tenant CRM makes "don't trust the server blindly" worth enforcing.
- **Recommended Fix (not applied):**

```
final safeId = RegExp(r'^[A-Za-z0-9_-]{1,128}$').hasMatch(mediaId)
    ? mediaId
    : sha256.convert(utf8.encode(mediaId)).toString();
final file = File(p.join(dir.path, 'wa_$safeId$ext'));
if (!p.isWithin(dir.path, file.path)) throw StateError('bad path');
```

Apply the same allow-list before building `ApiEndpoints.whatsappMedia`, and URL-encode path segments generally (`Uri.encodeComponent`).

- **References:** MASTG-TEST-0025 (Testing for Injection Flaws); CWE-22.

### FSA-004 – Attachment and call-recording URLs are launched with whatever scheme the server returns

- **Severity:** Medium – requires a hostile/compromised backend or an upstream integration (S3/CDN/Exotel URL fields) that an attacker can influence; result is arbitrary external-app launch rather than code execution.
- **OWASP Mobile Top 10:** M4 – Insufficient Input/Output Validation; M8
- **MASVS:** MASVS-PLATFORM-1
- **CWE:** CWE-939 (Improper Authorization in Handler for Custom URL Scheme), CWE-20
- **Location:** `lib/core/utils/attachment_link.dart:21-32` and `:53-66`; callers at `customer_activity_tabs.dart:514,850`, `lead_detail_screen.dart:1927,2277`, `task_detail_screen.dart:1101`, `followup_detail_screen.dart:1279`, `project_detail_screen.dart:1088`, `ops_task_detail_screen.dart:739`
- **Evidence:**

```
// attachment_link.dart:25-27
final parsed = Uri.tryParse(raw);
if (parsed == null) return null;
if (parsed.hasScheme) return parsed;          // any scheme accepted as-is
...
// attachment_link.dart:61-62
final opened = await launchUrl(uri, mode: LaunchMode.externalApplication) ||
    await launchUrl(uri);
```

The value originates in attachment rows ( `file` field) and Exotel call-log rows ( `recordingUrl` ) – server data with no scheme check.

- **Description:** `launchUrl` with a non-`http(s)` URI on Android dispatches an `ACTION_VIEW` intent for that scheme. `intent://…#Intent;…;end` , `sms:` , `tel:` , `market://` , `file://` , or another app's custom scheme can all be triggered by a tap the user believes opens a PDF. The manifest `<queries>` block only declares `https` , which stops `canLaunchUrl` probes, not the launch itself.
- **Attack Scenario:** (1) An attacker who can write an attachment URL (compromised backend, or a storage/CDN record they control) sets `file` to `sms:+91XXXXXXXXXX?body=…` or an `intent:` URI targeting a vulnerable exported activity on the device. (2) A salesperson taps "Open" on the Files tab. (3) The device launches the attacker-chosen target.
- **Impact:** Phishing/toll-fraud vectors and a pivot into other apps' exported components; scales to every user who taps the row.
- **Recommended Fix (not applied):**

```
Uri? attachmentUri(String url) {
  ...
  if (parsed.hasScheme) {
    return (parsed.scheme == 'https') ? parsed : null;   // allow-list
  }
  ...
}
```

Optionally also pin the host to the API origin or the known media bucket, and surface "Unsupported link" to the user.

- **References:** MASTG-TEST-0028 (Testing Deep Links / URL handling); url_launcher README warning on untrusted URLs; the repo's `SECURITY_AUDIT.md` §4.

### FSA-005 – No code obfuscation, no release build pipeline or documented release flags

- **Severity:** Low – defense in depth; Dart AOT code is still disassemblable (reFlutter/Blutter) and every string, route and URL is extractable with `strings libapp.so` .
- **OWASP Mobile Top 10:** M7 – Insufficient Binary Protections
- **MASVS:** MASVS-RESILIENCE-3
- **CWE:** CWE-656 (Reliance on Security Through Obscurity – inverse: none applied)
- **Location:** repository root (no CI/Fastlane/Codemagic/Makefile); `README.md:41-58` ; `.gitignore:44-48` (has rules for `app.*.symbols` / `app.*.map.json` , i.e. obfuscation *was anticipated*)
- **Evidence:**

```
$ grep -rnE "obfuscate|split-debug-info" --include=*.md --include=*.kts --include=*.yaml .
(no results outside SECURITY_AUDIT.md)
$ ls .github codemagic.yaml bitrise.yml fastlane .gitlab-ci.yml
(none exist)
```

`README.md:58` : `fvm flutter build apk --release --dart-define=SENTRY_RELEASE=clozr@1.0.0+1` – no obfuscation flags, no `API_BASE_URL` .

- **Description:** Release artefacts depend entirely on what a developer types. Combined with FSA-001 this is what lets a mis-configured build ship unnoticed.
- **Attack Scenario:** `unzip app-release.apk; strings lib/arm64-v8a/libapp.so | grep -E 'clozr|/api/v1'` lists the dev host, every endpoint, Sentry DSN and class names in seconds, speeding up API abuse and tampering.
- **Impact:** Faster reverse engineering; no direct data exposure by itself.
- **Recommended Fix (not applied):** Commit `tool/build_release.sh` (or a CI workflow) that always runs

```
fvm flutter build appbundle --release --obfuscate --split-debug-info=build/symbols \
  --dart-define-from-file=env/prod.json
```

and uploads `build/symbols` to Sentry ( `sentry-cli debug-files upload` ) so crashes stay symbolicated.

- **References:** MASTG-TEST-0038 (Obfuscation); Flutter docs "Obfuscate Dart code".

### FSA-006 – R8/ProGuard minification not enabled for the release build type

- **Severity:** Low – only the thin Kotlin/Java layer and plugin code are affected; Dart is untouched by R8.
- **OWASP Mobile Top 10:** M7
- **MASVS:** MASVS-RESILIENCE-3
- **CWE:** CWE-656
- **Location:** `android/app/build.gradle.kts:80-105`
- **Evidence:**

```
buildTypes {
    release {
        signingConfig = ...
        ndk { debugSymbolLevel = "FULL" }
        // no isMinifyEnabled / isShrinkResources / proguardFiles
    }
}
```

- **Description:** AGP's default for `release` is `isMinifyEnabled = false`. No `proguard-rules.pro` exists either.
- **Attack Scenario:** `jadx app-release.apk` shows plugin classes (flutter_secure_storage, open_file FileProvider) with original names, making hooking with Frida marginally easier.
- **Impact:** Minor.
- **Recommended Fix (not applied):**

```
release {
    isMinifyEnabled = true
    isShrinkResources = true
    proguardFiles(getDefaultProguardFile("proguard-android-optimize.txt"), "proguard-rules.pro")
}
```

  Test `flutter_secure_storage`, `open_file`, `in_app_update` and Sentry after enabling; add keep rules if needed.
- **References:** MASTG-TEST-0038.

### FSA-007 – Debug Dio logger dumps full response bodies, including the JWT pair

- **Severity:** Low – guarded by `kDebugMode`, so stripped from release; the risk is developers sharing logcat dumps that contain live refresh tokens for UAT/production accounts.
- **OWASP Mobile Top 10:** M6 – Inadequate Privacy Controls
- **MASVS:** MASVS-STORAGE-2
- **CWE:** CWE-532 (Insertion of Sensitive Information into Log File)
- **Location:** `lib/core/network/api_service.dart:39-54`
- **Evidence:**

```
if (kDebugMode) {
  // TEMPORARY: responseBody is ON ... including `/auth/login/` and
  // `/auth/login/refresh/`, whose bodies ARE the JWT pair.
  _dio.interceptors.add(PrettyDioLogger(
    requestBody: false,
    responseBody: true,
```

  Also `quotes_remote_ds.dart:184-192` prints full quote payloads inside `assert(() { ... }())` (debug-only).
- **Description:** The comment itself says this is temporary and dangerous. Debug builds point at `dev.clozr.tech` by default (FSA-001), and UAT testers often run debug builds, so refresh tokens for real UAT accounts land in logcat, which is world-readable to other apps on Android < 4.1 and to anyone the tester shares logs with.
- **Attack Scenario:** Tester pastes a logcat excerpt into a bug ticket → the `refresh_token` inside it is valid until server-side expiry → anyone reading the ticket mints access tokens.
- **Impact:** UAT/dev account takeover via shared logs; none in release.
- **Recommended Fix (not applied):** Set `responseBody: false`, or add a redacting `LogInterceptor` that masks `access_token` / `refresh_token` / `backup_codes` keys; remove the `TEMPORARY` block before tagging a release.
- **References:** MASTG-TEST-0003; OWASP Logging Cheat Sheet.

### FSA-008 – Client-generated `user_id` for member invites uses non-cryptographic `Random()`

- **Severity:** Low – only matters if the server trusts the client-chosen identifier for anything beyond a correlation key (see §4).
- **OWASP Mobile Top 10:** M10 – Insufficient Cryptography
- **MASVS:** MASVS-CRYPTO-1
- **CWE:** CWE-338 (Use of Cryptographically Weak PRNG)
- **Location:** `lib/features/people/infrastructure/data_sources/remote/people_remote_ds.dart:155-156` and `:617-619`
- **Evidence:**

```
await _api.post(ApiEndpoints.users, body: {
  'user_id': generateUuid(),
...
static String generateUuid() {
  final rng = Random();                      // not Random.secure()
  final bytes = List<int>.generate(16, (_) => rng.nextInt(256));
  final now = DateTime.now().microsecondsSinceEpoch;
```

- **Description:** Dart's default `Random` is a seeded PRNG; XOR-ing with the clock does not make it unpredictable. If the backend uses the supplied `user_id` as the primary key and does not reject collisions, a predictable id could be pre-registered or collided by another tenant admin.

- **Attack Scenario:** Depends on server behaviour (unverifiable here). Worst case: attacker predicts/collides a `user_id`, causing an invite to attach to an existing account.
- **Impact:** Low; mostly a hygiene issue.
- **Recommended Fix (not applied):** `final rng = Random.secure();` or add the `uuid` package. Ideally the server should generate the id.
- **References:** MASTG-TEST-0014 (Random Number Generation).

### FSA-009 – 2FA backup codes and OTP entry displayed without screenshot / app-switcher protection

- **Severity:** Low – requires malware with screen-capture permission or physical access during the moment the codes are shown.
- **OWASP Mobile Top 10:** M6
- **MASVS:** MASVS-PLATFORM-3
- **CWE:** CWE-200 (Exposure of Sensitive Information)
- **Location:** `lib/features/auth/presentation/screens/login_screen.dart:385-412` (`_backupCodesView`, `SelectableText(_backupCodes.join('\n'))`), 2FA code field at `:327-376`
- **Evidence:**

```
$ grep -rnE "FLAG_SECURE|secureFlag|setFlags|WindowManager" lib android ios
(no results)
```

- **Description:** One-time backup codes are the recovery credential for 2FA. They are rendered as selectable text (copyable to the clipboard) with no `FLAG_SECURE` on Android and no blur in the iOS app switcher; the login/2FA screen likewise.
- **Attack Scenario:** Screen-recording malware or an accessibility-abusing app captures the backup-code screen → attacker later bypasses 2FA with the stolen code.
- **Impact:** 2FA bypass for the affected user.
- **Recommended Fix (not applied):** Use a small platform channel or a package such as `flutter_windowmanager` / `screen_protector` to set `FLAG_SECURE` while the auth flow is on screen; show backup codes with an explicit "Copy" action rather than free selection; clear the clipboard after a timeout.
- **References:** MASTG-TEST-0010 (Sensitive Data in UI), MASTG-TEST-0011 (App Switcher snapshots).

### FSA-010 – HTTPS-only enforcement is an `assert`, which is stripped from release builds

- **Severity:** Low – Android `usesCleartextTraffic="false"` and iOS ATS still block plaintext, so this is a missing belt, not a missing floor.
- **OWASP Mobile Top 10:** M5 – Insecure Communication
- **MASVS:** MASVS-NETWORK-1
- **CWE:** CWE-617 (Reachable Assertion used as a security control)
- **Location:** `lib/app/bootstrap/app_bootstrap.dart:47-53`
- **Evidence:**

```
// Fail fast on an insecure production misconfiguration ...
assert(
  !ApiConfig.apiEnabled || ApiConfig.isBaseUrlSecure,
  'API_BASE_URL must use https:// ...',
);
```

- **Description:** `assert` bodies are removed by the AOT compiler in release/profile mode, so the "fail fast on production misconfiguration" never runs in production.
- **Attack Scenario:** A release built with `--dart-define=API_BASE_URL=http://…` would start normally; the OS network policy becomes the only barrier.
- **Impact:** Low in isolation.
- **Recommended Fix (not applied):** Replace with a real check: `if (ApiConfig.apiEnabled && !ApiConfig.isBaseUrlSecure) throw StateError(...)`.
- **References:** MASTG-TEST-0019 (Data Encryption on the Network).

### FSA-011 – Custom-scheme deep link `clozr://app` is hijackable and unverified; not declared on iOS

- **Severity:** Low – the router only navigates to screens behind the session gate and performs no sensitive action from the link; unauthenticated links are redirected to login and dropped.
- **OWASP Mobile Top 10:** M8
- **MASVS:** MASVS-PLATFORM-1
- **CWE:** CWE-939
- **Location:** `android/app/src/main/AndroidManifest.xml:41-50`; `lib/app/router/app_router.dart:132-161`; `ios/Runner/Info.plist` (no `CFBundleURLTypes`)
- **Evidence:**

```
<intent-filter>
    <action android:name="android.intent.action.VIEW" />
    <category android:name="android.intent.category.DEFAULT" />
    <category android:name="android.intent.category.BROWSABLE" />
    <data android:scheme="clozr" android:host="app" />
</intent-filter>
```

- **Description:** Any app can register `clozr://` and intercept links intended for Clozr (link hijacking). Since no sensitive action (login, reset, payment) is driven by the link, the exposure is limited to phishing-style redirection. The scheme is Android-only; iOS has no URL type, so the feature is inconsistent.
- **Attack Scenario:** Malicious app registers `clozr://app`, a user taps a link from a notification/email, the malicious app receives the path (e.g. `/operations/tasks/<id>`), leaking record ids and allowing a fake login prompt.
- **Impact:** Low.
- **Recommended Fix (not applied):** Move to verified App Links (`android:autoVerify="true"` with an `https://app.clozr.co.in/.well-known/assetlinks.json`) and Universal Links on iOS; keep the router guard (already present) and never trigger state-changing actions from a link.
- **References:** MASTG-TEST-0028.

### FSA-012 – Sentry DSN hardcoded; request URL and query string are sent with 5xx events

- **Severity:** Info – a DSN is a public write-only key by design; the only abuse is event spam/quota exhaustion. Query strings may contain search terms.
- **OWASP Mobile Top 10:** M6
- **MASVS:** MASVS-PRIVACY-1
- **CWE:** CWE-359
- **Location:** `lib/core/monitoring/sentry_config.dart:31-35` (DSN `https://c920…88`, redacted), `:114-130`
- **Evidence:**

```
return event.copyWith(
  request: SentryRequest(
    url: request.url,
    method: request.method,
    queryString: request.queryString,   // kept
    ... // data dropped, auth headers redacted
```

- **Description:** Good: body dropped, `Authorization`/cookies redacted, `sendDefaultPii` off, screenshots off, 4xx filtered. Residual: list endpoints are called with `search=<customer name>` style query parameters (server-search providers), so a 5xx on such a request ships a customer name to Sentry. The DSN can be extracted from the binary and used to flood the project.
- **Recommended Fix (not applied):** Drop or hash `queryString` in `_beforeSend`; enable Sentry's inbound filters / rate limits and consider `--dart-define`-only DSN with no default.
- **References:** Sentry "Scrubbing Sensitive Data"; MASTG-TEST-0003.

### FSA-013 – Raw exception text surfaced to users in toasts

- **Severity:** Info – the exceptions are local `PlatformException`s (file picker, `OpenFile`), not server stack traces; `AppError.fromDio` already maps server bodies through an allow-list of keys.
- **OWASP Mobile Top 10:** M6
- **MASVS:** MASVS-PLATFORM-3
- **CWE:** CWE-209
- **Location:** `lib/features/crm/presentation/screens/quote_detail_screen.dart:485`; `customer_activity_tabs.dart:160`; `task_detail_screen.dart:1119`; `followup_detail_screen.dart:1300`; `project_detail_screen.dart:1223`; `ops_task_detail_screen.dart:790`; `ticket_detail_screen.dart:1527`; `notes_thread.dart:293,459`; `attachment_link.dart:65`
- **Evidence:**

```
toast.showError('Could not open the PDF: $e');
```

- **Recommended Fix (not applied):** Show a fixed message and send `e` to `AppMonitoring.captureError`.

### FSA-014 – Generated `.flutter-plugins` committed with a developer's home-directory path

- **Severity:** Info
- **OWASP Mobile Top 10:** M8
- **CWE:** CWE-200
- **Location:** `.flutter-plugins:1-3` (`/Users/<developer>/.pub-cache/...`); `.gitignore:35` ignores `.flutter-plugins-dependencies` but not `.flutter-plugins`
- **Description:** The file header says "do not check into version control". It leaks a developer username and causes build-machine path mismatches. No secret content.
- **Recommended Fix (not applied):** Add `.flutter-plugins` to `.gitignore` and `git rm --cached` it.

### FSA-015 – `hive` 2.2.3 is upstream-abandoned; `open_file` 3.x legacy manifest behaviour

- **Severity:** Info – no published advisory for the locked versions; maintenance risk.
- **OWASP Mobile Top 10:** M2 – Inadequate Supply Chain Security
- **MASVS:** MASVS-CODE-3

- **Location:** `pubspec.yaml` ( `hive: ^2.2.3` , `open_file: ^3.5.11` ), `pubspec.lock`
- **Description:** See §5. The original Hive project is abandoned (community forks `hive_ce` / `hive_io` exist), so an encryption or corruption bug would not be fixed upstream. `open_file` 3.x historically required apps to justify `REQUEST_INSTALL_PACKAGES` on Play; 4.0.0 removed that check. The locked 3.5.11 manifest could not be fetched offline – verify the merged manifest (see §4).
- **Recommended Fix (not applied):** Plan a move to `hive_ce` (drop-in API) or to `drift` / `sqflite` + SQLCipher once the Flutter pin is lifted; check `build/app/intermediates/merged_manifests/release/AndroidManifest.xml` for `REQUEST_INSTALL_PACKAGES` and `<provider>` entries contributed by `open_file` ; move to `open_file` 4.x / `open_filex` when the toolchain allows.

## 4. Needs Manual Verification

These cannot be confirmed from the repository. Each should be checked before relying on the "no Critical findings" conclusion.

| # | What to verify | How |
|---|---|---|
| MV-1 | **Backend authorization per request.** The app gates UI with `ensurePermission` / `moduleAccessProvider` (client-side) and the code comments say the server enforces per request. Confirm every `/api/v1/**` endpoint checks role + tenant (object-level ownership), especially `PATCH /management/users/{id}/` , quote discount approvals, payment records, accounting postings. | Replay authenticated requests across two tenants/roles with Burp or `curl` ; confirm 403s. |
| MV-2 | **Invite `user_id` handling** (FSA-008). Does `POST /management/users/` reject a `user_id` that already exists, and is it ever used as an authorization key? | Read the DRF serializer/view; attempt a duplicate id. |
| MV-3 | **Refresh-token lifetime and rotation.** `AuthInterceptor._refresh` stores only the new `access` ; if SimpleJWT `ROTATE_REFRESH_TOKENS=True` with blacklisting is on, the app keeps using the *old* refresh token (functional break); if rotation is off, confirm the refresh TTL is short and `/auth/logout/` blacklists it. | Check `SIMPLE_JWT` settings; test logout → reuse old refresh token. |
| MV-4 | **Media / attachment URL authorization.** `attachment_preview.dart:55-56` loads `NetworkImage(uri)` with **no** bearer header, and `openAttachment` hands URLs to the OS browser. The S3/CDN links must therefore be short-lived signed URLs; if they are long-lived public URLs, any leaked link exposes customer files. | Inspect an attachment `file` value; fetch it from a logged-out curl after 1 hour. |
| MV-5 | **Whether `media_id` / attachment `file` values can be influenced by third parties** (FSA-003/004): WhatsApp inbound webhooks, Exotel recording callbacks, Meta Ads imports. If the server stores these verbatim, the "requires malicious backend" caveat weakens to "requires a malicious WhatsApp sender". | Review the webhook handlers' validation. |
| MV-6 | **Rate limiting / lockout** on `/auth/login/` , `/auth/login/2fa/` , `/auth/password-reset/` (all `AllowAny` per `docs-flutter/security_systems.md` ). | Check DRF throttling classes; brute-force test in UAT. |
| MV-7 | **Play App Signing & upload key custody.** `android/key.properties` and `*.jks` are gitignored and never appeared in the reachable history – confirm they are not in another branch/remote, a shared drive, or chat. | `git log --all` on the full remote; ask the release owner. |
| MV-8 | **Shipped artefacts.** The previous audit found a dev-pointed `app-release.aab` under `build/` . Confirm no such artefact was uploaded, and that the current Play/TestFlight builds point at `app.clozr.co.in` . | `unzip -p base.apk lib/arm64-v8a/libapp.so \| strings \| grep clozr` . |
| MV-9 | **Merged Android manifest** for permissions and providers contributed by plugins ( `open_file` , `file_picker` , `in_app_update` ), in particular `REQUEST_INSTALL_PACKAGES` and FileProvider paths. | Build once and read `build/app/intermediates/merged_manifests/release/AndroidManifest.xml` , or `aapt dump permissions` . |
| MV-10 | **Account deletion.** No in-app or documented self-service account/data deletion path exists in `lib/` or `docs-flutter/` . Google Play and Apple require one for apps with account creation. | Confirm the web app or a support process exists and is linked from the store listing. |
| MV-11 | **Sentry project settings**: data scrubbing, IP storage off, retention. | Sentry → Project Settings → Security & Privacy. |
| MV-12 | **iOS build settings** not in the repo: Xcode release scheme uses `Release` config ( `Runner.xcscheme` appears correct), bitcode off ( `ENABLE_BITCODE = NO` ), and the archive is built with `--obfuscate` . | Check the CI/archive command used by the iOS release owner. |

## 5. Dependency Review

Locked versions from `pubspec.lock`. No Git or path dependencies; no `dependency_overrides`. Advisory lookup performed via OSV / GitHub Advisory Database search on 2026-10-27; only `dio` has a historical advisory, which does not affect the locked version.

| Package | Version | Issue | Advisory | Recommendation |
|---|---|---|---|---|
| dio | 5.11.0 | None for this version. CRLF injection via attacker-controlled HTTP method affected `< 5.0.0`. | CVE-2021-31402 / GHSA-9324-jv53-9cc8 (fixed in 5.0.0) | Keep; already patched. |
| pretty_dio_logger | 1.4.0 | Logs response bodies in debug (FSA-007). | – | Disable `responseBody` or redact. |
| flutter_secure_storage | 9.2.4 | Correctly configured with `encryptedSharedPreferences: true`. 10.x available but blocked by the Flutter 3.24.5 pin. | none found | Upgrade when the toolchain pin lifts. |
| hive / hive_flutter | 2.2.3 / 1.1.0 | Upstream abandoned; no encryption used (FSA-002). | none | Migrate to `hive_ce` (maintained fork) and enable `HiveAesCipher`. |
| open_file | 3.5.11 | Legacy 3.x line; `REQUEST_INSTALL_PACKAGES` check removed only in 4.0.0; merges a FileProvider. | none | Verify merged manifest (MV-9); move to 4.x / `open_filex` when possible. |
| url_launcher | 6.3.1 | Used with unvalidated schemes (FSA-004) – app-side issue. | none | Allow-list schemes. |
| file_picker | 8.3.7 | Uses SAF; no storage permissions declared (good). | none | Keep. |
| path_provider | 2.1.5 | – | none | Keep. |
| sentry_flutter / sentry_dio | 8.14.2 | Held at 8.x due to toolchain pin; well configured. | none | Upgrade to 9.x with the toolchain. |
| in_app_update | 4.2.5 | Android-only; guarded by `Platform.isAndroid`. | none | Keep. |
| go_router | 14.8.1 | – | none | Keep. |
| flutter_riverpod | 2.6.1 | – | none | Keep. |
| connectivity_plus, package_info_plus, emoji_picker_flutter, flutter_svg, fl_chart, intl | various | No security relevance found. | none | – |
| **Toolchain** | Flutter 3.24.5 / Dart 3.5.4 (pinned via `.fvmrc`) | Pin blocks security-relevant upgrades of `flutter_secure_storage`, `sentry_flutter`, `open_file`, `in_app_update`. Dart/Flutter pub client CVE-2026-27704 (malicious package archive extraction) is fixed only in Dart 3.11 / Flutter 3.41 – this affects **developer machines running `pub get`**, not the shipped app. | CVE-2026-27704 | Plan the toolchain migration described in `README.md`; until then only add packages from trusted publishers. |
| Android | `minSdk 24`, `targetSdk 36`, `compileSdk 36` | Modern floor; fine. | – | – |
| iOS | Deployment target 13.0 | Acceptable; consider 15+ to drop devices without current patches. | – | – |

## 6. Positive Findings

Keep these; they are easy to regress.

- **Tokens in platform secure storage, never in Hive** – `lib/core/storage/token_storage.dart:10-12` with `AndroidOptions(encryptedSharedPreferences: true)`; `RememberedEmail` uses the same store and never stores a password.

- **No disabled TLS validation anywhere** – no `badCertificateCallback`, `HttpOverrides`, `IOHttpClientAdapter`, or `SecurityContext` in `lib/`. `android:usesCleartextTraffic="false"` set explicitly; no `NSAppTransportSecurity` exceptions in `Info.plist`; no `network_security_config.xml` weakening defaults.
- **No hardcoded third-party secrets** – repo-wide and git-history grep for AWS/Google/Stripe/OpenAI/JWT/private-key patterns returned nothing; no `.env`, keystore, `key.properties`, `google-services.json`, or service-account file was ever committed in the reachable history. `android/.gitignore` excludes `key.properties`, `*.jks`, `*.keystore`.
- **Bearer header not sent to auth endpoints** – `AuthInterceptor._anonymousPaths` (`auth_interceptor.dart:49-61`) keeps the token off login/2FA/reset/refresh calls; terminal 401 codes (`session_revoked`, `user_inactive`, …) clear tokens without attempting refresh (`:77-86`).
- **Single-flight refresh with a bare retry client** – prevents recursion and token-refresh storms (`auth_interceptor.dart:44-45,99`).
- **Logout clears everything** – server-side blacklist call, then `AppCache.clearAll()` *before* token deletion, `UserDirectory.reset()`, Sentry user detached, provider graph invalidated (`auth_providers.dart:190-273`). New sessions wipe the cache first (`:207-215`).
- **Sentry PII controls** – `sendDefaultPii=false`, screenshots and view hierarchy off, request `data` dropped, auth/cookie/CSRF headers redacted, only user id + org id attached, expected failures filtered (`sentry_config.dart:69-151`).
- **Debug logging is debug-only** – `PrettyDioLogger` under `kDebugMode`; quote payload prints inside `assert`. No `print()` in release paths.
- **Minimal permissions** – Android declares only `INTERNET`; attachments use SAF so no storage/media permissions (`AndroidManifest.xml:8-13`). iOS declares only `NSPhotoLibraryUsageDescription`.
- **Task hijacking mitigated** – `android:taskAffinity=""` and `launchMode="singleTop"` on `MainActivity`.
- **Platform code is inert** – `MainActivity.kt` and `AppDelegate.swift` are stock; no custom `MethodChannel` handlers to inject into.
- **Quote PDF download sanitises its filename** – `quote_detail_screen.dart:474` strips everything but `[A-Za-z0-9_-]` (the model FSA-003 should follow).
- **Password field uses `obscureText`**, and `isBaseUrlSecure` only whitelists loopback/emulator hosts for cleartext.
- **Client-side permission gates are explicitly documented as UX only** (`auth_providers.dart:316-317`, `permission_guard.dart`) with the backend named as the enforcement point.
- **No WebView, no in-app purchase, no biometric gate, no raw SQL, no Firebase/Supabase** – whole classes of common Flutter vulnerabilities are absent by design.

## 7. Prioritized Remediation Roadmap

**Immediately (today)**

1. Remove the `defaultValue: 'https://dev.clozr.tech'` from `ApiConfig.baseUrl`, fail closed in release when it is empty, and verify the currently published Play/TestFlight builds point at `app.clozr.co.in` (FSA-001, MV-8). Delete any stale `build/` artefacts.
2. Set `android:allowBackup="false"` plus `dataExtractionRules` / `fullBackupContent` excluding app files (FSA-002, part 1).
3. Restrict `attachmentUri` to `https` (FSA-004) and sanitise `media_id` before using it as a filename or path segment (FSA-003).
4. Turn `responseBody` back off in the debug logger (FSA-007).

**Short term (this sprint)** 5. Commit a release build script/CI job that always passes `API_BASE_URL`, `SENTRY_ENVIRONMENT`, `--obfuscate --split-debug-info`, and uploads symbols (FSA-005); enable R8 (FSA-006). 6. Encrypt Hive boxes with a key stored in `flutter_secure_storage`; stop caching the full `auth` profile (FSA-002, part 2). 7. Replace the HTTPS `assert` with a runtime check (FSA-010); switch `Random()` to `Random.secure()` (FSA-008); drop `queryString` in Sentry `_beforeSend` (FSA-012). 8. Work through §4 with the backend team – especially MV-1, MV-3, MV-4, MV-6.

**Long term (hardening)** 9. `FLAG_SECURE`/app-switcher blur on the login, 2FA and backup-code screens (FSA-009). 10. Verified App Links / Universal Links instead of the custom scheme (FSA-011). 11. Lift the Flutter 3.24.5 pin so `flutter_secure_storage` 10.x, `sentry_flutter` 9.x, `open_file` 4.x and Dart ≥ 3.11 can be adopted; migrate `hive` → `hive_ce` (FSA-015). 12. Consider Play Integrity / App Attest and basic root/hook detection as defence in depth for a CRM carrying payment and invoice data; implement or link an account-deletion path (MV-10).

## 8. Appendix

### Commands / greps used (all read-only)

```
git branch --show-current; git log --oneline | head; git branch -a
find lib android ios assets -type f | sort
cat pubspec.yaml; awk '...' pubspec.lock           # locked versions
git grep -nIiE 'AIza[0-9A-Za-z_-]{20,}|sk_live|sk_test|sk-[A-Za-z0-9]{20}|AKIA[0-9A-Z]{12,}|-----
BEGIN|service_role|client_secret|private_key|api[_-]?key|secret[_-]?key'
git log --all -p -- . ':!pubspec.lock' ':!docs-flutter' | grep -E '^\+' | grep -iE '<same patterns>|storePassword=|keyPassword=|eyJ[A-Za-z0-9_-]
{20,}\.'
git log --all --name-only --pretty=format: | sort -u | grep -iE '\.env|\.jks|\.keystore|\.p12|\.pem|key\.properties$|google-services|GoogleService-
Info|service[-_]account|firebase_options'
grep -rnE "\bprint\(|debugPrint\(|\blog\(|Logger\(" lib
grep -rn "http://" lib
grep -rnE "badCertificate|HttpOverrides|onHttpClientCreate|IOHttpClientAdapter|SecurityContext" lib
grep -rnE "launchUrl|launchUrlString|canLaunchUrl|OpenFile\.open" lib
grep -rn "Clipboard\." lib
grep -rnE "obscureText|autocorrect|enableSuggestions|autofillHints" lib/features/auth
grep -rnE "Random\
(|md5|sha1|sha256|rawQuery|WebView|InAppWebView|HiveAesCipher|encryptionCipher|FLAG_SECURE|local_auth|in_app_purchase|SharedPreferences|RootBeer|ja
ilbreak|Integrity|AppCheck" lib pubspec.yaml android ios
grep -rnE "obfuscate|split-debug-info|minifyEnabled|isMinifyEnabled|shrinkResources|proguard" --include=*.md --include=*.kts --include=*.pro --
include=*.yaml .
grep -rn "AppCache.put(" lib
grep -rnE "source: (git|path)" pubspec.lock; grep -n dependency_overrides pubspec.yaml
grep -n "IPHONEOS_DEPLOYMENT_TARGET|ENABLE_BITCODE|CODE_SIGN|DEVELOPMENT_TEAM" ios/Runner.xcodeproj/project.pbxproj
```

Web lookups (no requests to any Clozr host): OSV/GitHub Advisory search for `dio`; pub.dev changelog for `open_file`; `open_file` upstream Android manifest; maintenance status of `hive`.

### Files reviewed in full or in relevant part

`pubspec.yaml`, `pubspec.lock`, `.gitignore`, `.flutter-plugins`, `.fvmrc`, `.vscode/settings.json`, `README.md`, `SECURITY_AUDIT.md` (prior in-repo audit, used as a map and independently re-verified), `web/index.html`; `lib/main.dart`, `lib/app/bootstrap/app_bootstrap.dart`, `lib/app/router/app_router.dart`, `lib/core/config/api_config.dart`, `lib/core/network/{api_service,auth_interceptor,api_endpoints,app_error}.dart`, `lib/core/storage/{token_storage,app_cache,remembered_email}.dart`, `lib/core/monitoring/{sentry_config,app_monitoring}.dart`, `lib/core/utils/attachment_link.dart`, `lib/core/widgets/attachment_preview.dart`, `lib/core/services/in_app_update_service.dart`, `lib/features/auth/**` (providers, remote DS, repository, login screen, guards), `lib/features/messages/presentation/screens/chat_screen.dart`, `lib/features/messages/infrastructure/**`, `lib/features/crm/infrastructure/data_sources/remote/{quotes,exotel}_remote_ds.dart`, `lib/features/crm/presentation/screens/quote_detail_screen.dart`, `lib/features/crm/application/providers/{lead_whatsapp,lead_call}_providers.dart`, `lib/features/people/infrastructure/data_sources/remote/people_remote_ds.dart`, `lib/features/accounting/presentation/screens/acc_document_print_screen.dart`, `lib/features/notifications/application/notification_links.dart`; all 29 `AppCache.put` call sites; `android/app/src/{main,debug,profile}/AndroidManifest.xml`, `android/app/build.gradle.kts`, `android/build.gradle.kts`, `android/gradle.properties`, `android/key.properties.example`, `android/.gitignore`, `android/app/src/main/kotlin/com/nexotech/clozrapp/MainActivity.kt`; `ios/Runner/Info.plist`, `ios/Runner/AppDelegate.swift`, `ios/Podfile`, `ios/Flutter/{Debug,Release}.xcconfig`, `ios/Runner.xcodeproj/project.pbxproj` (build settings); `docs-flutter/security_systems.md`, `docs-flutter/API_INTEGRATION_GUIDE.md`, `docs-flutter/HIVE implementation.md`, `test/**` (credential grep).

### Checklist coverage (A–N)

- **A. Secrets:** No hardcoded third-party keys, no `.env` asset, no service-account/keystore/certs in repo or reachable history, no test/backdoor credentials. Sentry DSN present (public-by-design, FSA-012). `--dart-define` is used for config, not secrets. **No issues beyond FSA-012.**
- **B. Backend/BaaS:** No Firebase/Supabase; custom Django REST backend. Rules not in repo → §4 MV-1/2/3/5/6.
- **C. Insecure storage:** FSA-002 (Hive + backup). Secure storage correctly used for tokens. No external storage writes; temp-dir writes in FSA-003 and the quote PDF path.
- **D. Network:** No cert bypass, no cleartext, no pinning (acceptable for this risk tier; note in roadmap). FSA-010. Tokens never in query strings. Bearer attached only to the single API Dio with a fixed base URL.
- **E. Auth/session:** JWT + 2FA, server-side logout, forced logout on terminal codes, no client JWT decoding, no biometric gate. MV-3 (rotation), MV-6 (rate limiting). FSA-009.
- **F. Business logic/payments:** No IAP. Quote totals, discounts and payment records are posted to and validated by the server (comments in `quotes_remote_ds.dart:170-176` confirm server derives `total_amount`); client permission checks are UX only. → MV-1.
- **G. Platform/IPC:** FSA-011 (custom scheme). Only `MainActivity` exported (launcher). No custom platform channels. Task affinity hardened. No `FileProvider` of the app's own; plugin-contributed one → MV-9.
- **H. WebView:** None used. **No issues.**
- **I. Code protection:** FSA-005, FSA-006. No root/tamper detection (noted as long-term).
- **J. Logging/leakage:** FSA-007, FSA-009, FSA-012, FSA-013. No analytics SDK other than Sentry.
- **K. Input validation:** FSA-003 (path), FSA-004 (scheme). No SQL, no HTML/Markdown rendering of untrusted content.
- **L. Cryptography:** FSA-008. No home-made encryption, no hardcoded keys/IVs. **Otherwise no issues.**

- **M. Supply chain:** §5, FSA-015. No git/path deps or overrides.
- **N. Privacy/compliance:** Minimal permissions; only ids to Sentry; MV-10 (account deletion); MV-11.

### Assumptions and limitations

- Static review only; no device, no build, no network requests to Clozr infrastructure. Release artefacts referenced by the prior in-repo audit (`build/app-release.aab`) are not present in this checkout and were not re-examined.
- The clone contains a single branch (`fix/uat-crm-common-parity`, 50 commits). Other branches, tags and any history beyond that depth could not be searched for secrets.
- Server behaviour (authorization, throttling, token rotation, URL signing, webhook validation) is inferred from client code and `docs-flutter/*.md` only, and is listed in §4 rather than asserted.
- Advisory lookup used public databases on the audit date; absence of a published advisory is not proof of absence of vulnerabilities.
- Severity ratings follow the table in the engagement brief and are justified per finding; the prior in-repo audit rated FSA-003 "High" – this report rates it Medium because exploitation requires control of the backend and the write primitive is confined to the app sandbox.